

# FBLA App - How Everything Works (Presentation Guide)

## 1) Project Overview

The FBLA app is a React Native app built with **Expo + Expo Router**. Its goal is to help FBLA members manage:

- account/profile setup
- reminders by competition scope (National/State/Regional)
- dues tracking progress
- quick weekly overview on the home screen

## 2) Tech Stack

- **Framework:** Expo (React Native)
- **Routing:** expo-router (file-based routing)
- **Language:** TypeScript
- **Storage:** AsyncStorage (local device storage)
- **UI Libraries:**
  - react-native-calendars
  - @react-native-picker/picker
  - @expo/vector-icons

## 3) Folder Structure (Key Parts)

- app/\_layout.tsx : Root layout + providers
- app/(tabs)/\_layout.tsx : Auth gating + tab navigation
- app/(tabs)/landing.tsx : First entry screen (unauthenticated)
- app/(tabs)/auth-choice.tsx : Login vs Signup choice
- app/(tabs)/login.tsx : Existing user login
- app/(tabs)/signup.tsx : New account + initial profile data
- app/(tabs)/profile-setup.tsx : Required setup if profile incomplete
- app/(tabs)/index.tsx : Home tab (weekly summary)
- app/(tabs)/reminders.tsx : Reminder calendar and CRUD
- app/(tabs)/dashboard.tsx : Profile snapshot + dues progress + logout

- `app/(tabs)/other.tsx` : Placeholder future features list
- `app/(tabs)/ThemeContext.tsx` : Custom light/dark color context
- `lib/authStorage.ts` : Core local auth + user data persistence

## 4) App Startup and Navigation

### Root layout

`app/_layout.tsx` wraps the app in:

- custom `ThemeProvider`
- navigation theme provider
- stack routes

It declares `(tabs)` as the main app and keeps a modal route available.

### Tab layout + route protection

`app/(tabs)/_layout.tsx` determines the app state:

- `loading`
- `unauthenticated`
- `needs-setup`
- `ready`

Based on state, it redirects users:

- `unauthenticated` -> `/landing`
- `profile incomplete` -> `/profile-setup`
- `ready` -> normal tabs ( `Home` , `Reminders` , `Dashboard` , `Other` )

Important detail: it re-checks user state every second ( `setInterval` ) so auth/profile changes appear immediately.

## 5) Authentication and User Data

All user data is stored locally in AsyncStorage via `lib/authStorage.ts` .

### Storage keys

- `@fb1a_users_v1` -> database of all users
- `@fb1a_current_user_v1` -> username of the active session

### User record model

Each user contains:

- `username` , `password` , `displayName`
- `reminders` grouped by:
  - FBLA National
  - FBLA State
  - FBLA Regional
- `profile` fields:
  - `state`, `school`, `chapterName`, `gradDate`
  - `duesTotal`, `duesPaid`
- `profileComplete`

## Core auth functions

- `signUpUser(...)` creates user + empty reminders + starter profile
- `loginUser(...)` verifies credentials and sets current user
- `logoutUser()` clears current session
- `getCurrentUser()` reads active user
- `setCurrentUserProfile(...)` updates profile fields
- `setCurrentUserReminders(...)` writes reminder data

## 6) Screen-by-Screen Behavior

### Landing and Auth Choice

- Landing introduces app value and routes to auth choice.
- Auth choice sends user to either Login or Signup.

### Login

- Validates non-empty fields.
- Calls `loginUser` .
- On success routes to `/` , then tab layout handles correct state.

### Signup

- Collects account + profile fields in one flow.
- Uses state picker + calendar date picker.
- Creates account with `signUpUser` , then saves profile and marks `profileComplete: true` .

## Profile Setup

- Fallback flow for users whose profile is incomplete.
- Blocks access to full app until required fields are filled and saved.

## Home (`index.tsx`)

- Loads current user and reminders.
- Computes upcoming reminders within next 7 days.
- Shows:
  - greeting with display name
  - upcoming reminder count
  - dues progress percentage
  - list of upcoming items

## Reminders (`reminders.tsx`)

- Lets user choose reminder scope: National/State/Regional.
- Calendar behavior:
  - tap day -> view reminders for that date
  - long press day -> open add-reminder mode
- Supports adding and deleting reminders.
- Persists changes with `setCurrentUserReminders`.

## Dashboard

- Shows read-only profile snapshot.
- Shows dues progress bar ( `duesPaid / duesTotal` ).
- Plus button opens modal to increment total dues and paid amount.
- Logout button clears session and returns to auth flow.

## Other

- Displays planned expansion features (quizzes, directory, goals, badges, internships, conference mode).

## 7) Theme System

`ThemeContext.tsx` defines custom light/dark palettes and stores selected theme in `AsyncStorage` ( `@theme` ).

Current behavior:

- Theme state and colors are available globally through `useTheme()` .
- Most current screens use direct color constants (FBLA blue palette), so the custom theme is partially integrated.

## 8) Data Flow Summary

1. User signs up or logs in.
2. Current username is stored as active session.
3. Tab layout checks user state and routes accordingly.
4. Screens load user/reminder data from AsyncStorage.
5. User actions (add reminder, update dues, profile edits) write back to storage.
6. Polling refresh (every second in key screens/layout) keeps UI synced.

## 9) What Makes This App Strong for FBLA Use

- Structured by real FBLA levels (National/State/Regional)
- Practical student workflow (profile, deadlines, dues)
- Immediate value through weekly summary and calendar reminders
- Clear extension path via the `other` section

## 10) Talking Points for Presentation

- 💎 This app uses local-first storage, so it works without a backend server.💎
- 💎 Route protection ensures users cannot access member features before authentication and profile setup.💎
- 💎 Reminder data is categorized by FBLA scope to match real competition planning.💎
- 💎 Dashboard turns dues into a visual progress bar for quick chapter tracking.💎
- 💎 The codebase is modular: auth logic is centralized in `lib/authStorage.ts` , while each screen handles one clear responsibility.💎

## 11) Current Limitations (Honest Evaluation)

- Credentials are stored locally (not encrypted auth service).
- Data is per-device (no cloud sync between devices).
- Frequent 1-second polling is simple but not the most efficient approach.
- Theme context exists but is not yet fully applied to all UI components.

## 12) Next Step Upgrades

- Move auth/data to a backend (Firebase/Supabase) with secure authentication.

- Replace polling with event-driven updates/state management.
- Fully connect UI to `ThemeContext` for complete dark/light support.
- Add push notifications for reminders.
- Implement features listed on the `other` tab.